2025 Third International Conference on Networks, Multimedia and Information Technology (NMITCON) | 979-8-3315-1308-5/25/$31.00 ©2025 IEEE | DOI: 10.1109/NMITCON65824.2025.11187640

Secure by Design: Strategic Approaches to Infrastructure as Code

1st Josephine Eskaline Joyce
IBM India Private Limited
Bangalore, India
jojustin@in.ibm.com

2nd Shikha Maheshwari
IBM India Private Limited
Bangalore, India
shikha.mah@in.ibm.com

3rd Deepika Kothamasu
IBM India Private Limited
Bangalore, India
deepika.kothamasu@in.ibm.com

***Abstract*—Securing Infrastructure as Code (IaC) is a critical endeavor in today's dynamic and rapidly evolving technological landscape. IaC, with its emphasis on automating and codifying infrastructure deployment and management, introduces unique security challenges that require careful consideration and proactive measures. The overarching goal is to ensure the integrity, confidentiality, and availability of digital assets managed through IaC. This paper explores the comprehensive measures required to ensure the robust security of digital infrastructures managed through IaC. This study discusses the anatomy of a secure IaC environment, with an emphasis on the need for a proactive security stance throughout the IaC development life cycle. This paper also details the list of tools to be used at various phases of the infrastructure setup. With the proposed framework, it is noted that the security posture is improved, along with the reduction in effort to maintain IaC assets securely**

***Index Terms*—Infrastructure as Code, Shift-left Security, Vulnerability, Infrastructure, automation, Terraform**

I. Introduction

In the rapidly evolving landscape of digital infrastructure, the advent of IaC [1] has emerged as a transformative force, empowering organizations to provision and manage their IT resources with unprecedented speed and efficiency. As this paradigm shift accelerates, complexity increases in securing infrastructure, presenting key challenges like Hardcoded Secrets and Credentials [2], Misconfiguration & Insecure Defaults [3], Lack of Access Control & Role Separation [3], Insecure CI/CD Pipelines [4], Tooling Fragmentation [5].Addressing the security challenges associated with IaC, there is a need for a robust approach to security that safeguards digital assets against an array of potential threats [6]. Strong governance, when combined with the right tools and practices, becomes essential in ensuring that infrastructure remains secure, consistent, and resilient [7]. This article,takes a holistic view of IaC security—beginning with an exploration of its underlying structure to understand what needs protection, followed by key strategies to mitigate risks, and concluding with a practical framework that brings governance and security together to support secure IaC implementation throughout its lifecycle.

II. Related Work

The fundamental concept of IaC [8] lies in treating infrastructure configurations as code, allowing development and operations teams to collaborate seamlessly and deploy infrastructure changes with speed, consistency, and repeatability. While IaC revolutionizes the way organizations manage and deploy their digital infrastructure, it introduces a unique set of challenges in terms of security [1].

A. Related Work:

Paper [9], focuses on the IaC tools with code smells identification details. The focus of the research work presented in [10], is the anti-patterns of IaC development. It does not focus on the secure IaC. Paper [11] performs the research on IaC vulnerability and recommendations to detect and fix them. This paper provides in-depth details on the need for static analysis and how it helps in identifying the vulnerabilities in IaC code. Research [12] focuses on the development practices and identifies the challenges related to secrets management and static analysis. Research work presented in [13] details code smells and Security Linter as part of the static analysis of the IaC code base. The threats focused on in this paper are only on the IaC code. The research [14], focuses on the code smells and their impact. Code smells is the hardcoding of secrets, such as passwords, API keys, and tokens, directly within the scripts and compromises security. IaC scripts need access to passwords, keys, and tokens; this is considered one of the top challenges. One of the primary challenges in IaC security stems from misconfigurations. The automated nature of IaC deployments can lead to errors in configuration files, potentially exposing vulnerabilities [2] or creating security gaps. Securing digital assets requires meticulous attention to detail to ensure that the configurations align with security best practices. Paper [6] focuses on best practices of IaC and emphasizes the code smells leading to incremental system weakness. This study also provides the employability of threat models and their importance in the development stage and suggests considering threat models as one of the preferred tools for securing IaC assets. This [15] study proposes a DevSecOps framework to maintain secure, integral, and self-healed applications designed with IaC. This framework minimizes the required skill set and time for mastering various approaches. Based on a thorough examination of relevant literature, it is evident that the previous studies focus on the IaC scripts, code smells related to the scripts, and vulnerabilities. There hasn't been previous research that addresses the IaC security holistically, including development scripts, code commit process automated policy verification, static and dynamic analysis, threat detection, and security tests like pen testing.

B. Our Contribution

Our main contributions can be summarized as follows:

- Anatomy of Secure IaC - Provides the details on the anatomy of Secure IaC that provides the layered architecture of what makes secure IaC possible.
- Key Strategies for Securing IaC - A holistic approach for

979-8-3315-1308-5/25/$31.00 © 2025 IEEE

Authorized licensed use limited to: Atlantic Technological University (ATU). Downloaded on December 14,2025 at 21:16:13 UTC from IEEE Xplore. Restrictions apply.

securing IaC along with the methodology and tools involved.

- Secure IaC Framework - The framework acts as a governance overlay across both the anatomy and lifecycle phases.

The paper is structured as follows: Section III covers the details of the anatomy of IaC, Section IV introduces the key strategies to be followed for securing IaC, Section V explores the secure IaC framework, Section VI provides the results and the detailed discussions, and Section VII concludes the study.

III. Secure IaC

A. *Anatomy of Secure IaC:*

The anatomy of Secure IaC refers to the essential components, practices, and principles that make IaC secure by design and resilient by default. "Fig. 1" provides the detailed breakdown of various layers in securing IaC.

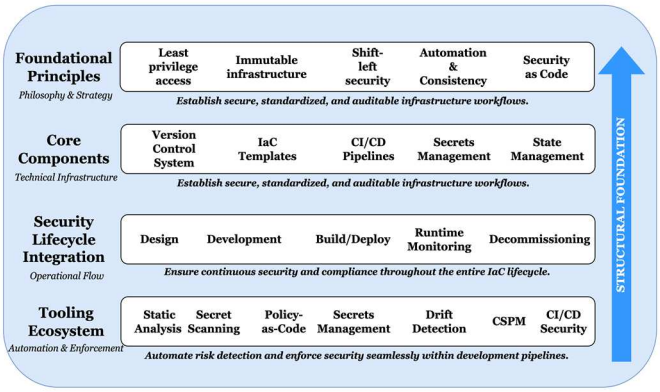

Fig. 1. Anatomy of Secure IaC

Following is the detailed breakdown of the Anatomy of Secure IaC across its four key layers.

1. Foundational Principles: These are the core security philosophies that shape how IaC is designed and operated. This layer ensures that security is embedded at a cultural and architectural level. Key elements are

- Least Privilege Access [16]: Limit access to only what is strictly necessary.
- Immutable Infrastructure: Replace rather than patch infrastructure to prevent drift.
- Shift-Left Security: Integrate security checks early in the development process.
- Security as Code [17]: Encode security policies and compliance rules in version-controlled files.

2. Core Components: This layer includes the essential technical building blocks that enable secure IaC workflows. This layer provides the technical foundation to build, store, and apply infrastructure definitions securely. Key elements are

- Version Control System (VCS): Centralized code repository with access control (e.g., GitHub, GitLab).
- CI/CD Pipelines: Automated workflows for testing, validating, and deploying IaC.
- Secrets Management: Secure storage and retrieval of secrets [18] (e.g., Vault, AWS Secrets Manager).
- State Management: Safe handling of infrastructure state files with encryption and locking.

3. Security Lifecycle Integration: This layer integrates security into every stage of the infrastructure lifecycle—from design to decommissioning. This layer ensures that security isn't an afterthought but a continuous process. Key elements are

- Design: Define secure architecture with threat modeling.
- Development: Apply secure coding practices and peer reviews.
- Build and Deploy: Automate validation and compliance checks.
- Runtime Monitoring: Detect and correct configuration drift or violations.
- Decommissioning: Securely retire resources and revoke access.

4. Tooling Ecosystem: This layer includes the tools that automate, enforce, and monitor security throughout the IaC pipeline. This layer empowers the organization to scale security through automation. Key elements are

- Static Analysis Tools [19]: tfsec, Checkov, and, KICS for scanning IaC code.
- Secret Scanners: TruffleHog and Gitleaks for detecting exposed credentials.
- Policy-as-Code [20] (PaC) Tools:OPA and Sentinel for enforcing security rules.
- CI/CD Integrations: GitHub Actions, Jenkins, and, and GitLab CI for enforcing gates and tests.

Following are the outcomes of Secure IaC Anatomy:

- Reduced configuration drift and misconfiguration risk
- Prevention of data exposure and access misuse
- Early detection of vulnerabilities and insecure patterns
- Enhanced system reliability and security posture

The anatomy of secure IaC forms the foundation for security practices to be applied consistently across environments.

IV. Key Strategies for Securing IaC

Securing the IaC lifecycle requires a proactive, end-to-end approach that embeds security at every phase. By adopting targeted security strategies, organizations can build and maintain resilient, compliant, and secure cloud infrastructure.

A. Design Phase— Secure by Design: Goal of this phase is to build security into the architecture before a single line of code is

TABLE I. Tools for Design

Tools	Purpose
ThreatMapper / ThreatModeler	Threat modeling for IaC and cloud systems
Open Policy Agent (OPA)	Define PaC for secure infrastructure design
NIST Cybersecurity Framework, CIS Benchmarks	Baseline frameworks for secure architecture planning
Well-Architected Tool (IBM, AWS, Azure)	Review design compliance with cloud best practices

a) Key Strategies:

- Threat modeling: Identify and mitigate risks in cloud and infrastructure designs early.
- Architecture review: Include security experts in design reviews to spot insecure design patterns.
- PaC planning: Design policies that enforce security through tools like OPA/Gatekeeper.

b) Tools and Frameworks: "Table. I" refers to the tools and frameworks involved during the design phase.

Authorized licensed use limited to: Atlantic Technological University (ATU). Downloaded on December 14,2025 at 21:16:13 UTC from IEEE Xplore. Restrictions apply.

B. Development Phase— Shift Left on Security: Goal of this phase is to prevent vulnerabilities in IaC templates and ensure secure coding practices.

a) Key Strategies:

- Static Analysis (SAST) tools for IaC: Use tools to catch misconfigurations in Terraform, CloudFormation, etc.
- Secrets scanning: Prevent hardcoding of secrets.
- Code reviews with security checklists: Ensure peer reviews include a security validation step.
- Version control and branch protections: Enforce PR checks, code reviews, and protected branches to prevent unauthorized changes.

b) Tools and Frameworks: "Table. II" refers to the tools and frameworks involved during the development phase.

TABLE II. TOOLS FOR DEVELOPMENT

Tools	Purpose
Checkov, tfsec, KICS	SAST for Terraform, CloudFormation, Kubernetes
tfsec	Terraform security scanning
KICS (Keep Infrastructure as Code Secure)	Scan IaC files for vulnerabilities
pre-commit + PaC hooks	Enforce security checks before code commits
Gitleaks	Secrets detection in repositories

C. Deployment Phase— Guardrails and Continuous Validation: Goal of this phase is to ensure secure provisioning and enforce runtime compliance.

a) Key Strategies:

- PaC enforcement at CI/CD: Block insecure IaC before deployment.
- Immutable infrastructure: Prefer replace-over-modify to avoid configuration drift or unintended exposure.
- Secure state file management: Encrypt and protect Terraform state files; store them in secure backends with access controls.
- Identity and permissions validation: Ensure least privilege policies [16] are validated pre-deploy.

b) Tools and Frameworks: "Table. III" refers to the tools and frameworks involved during the deployment phase.

TABLE III. TOOLS FOR DEPLOYMENT

Tools	Purpose
OPA (Gate-keeper/Conftest)	Policy enforcement during CI/CD
Terraform Sentinel / HashiCorp Sentinel	PaC for Terraform provisioning
Vault / IBM Cloud Secrets Manager	Secure secret injection and management
SOPS (Secrets OPerationS)	Encrypt secrets within IaC files securely
Infrastructure State Backends (S3, Azure Blob, GCS)	Secure and encrypted state file management

D. Drift Management Phase— Monitor and Remediate: Goal of this phase is to detect and remediate unauthorized or unintentional infrastructure changes.

a) Key Strategies:

- Drift detection [21] tools: Use tools to detect changes outside IaC.
- Automated remediation: Trigger alerts or rollbacks for drift detection and remediation.
- Audit logging: Continuously monitor and log changes to infrastructure and sensitive resources.

b) Tools and Frameworks: "Table. IV" refers to the tools and frameworks that can be used to manage drifts.

TABLE IV. TOOLS FOR DRIFT MANAGEMENT

Tools	Purpose
Driftctl	Detect drift between deployed infrastructure and IaC
Terraform Plan + CI Diff Checks	Detect unauthorized infrastructure changes
AWS Config / Azure Resource Graph	Detect configuration drift in cloud services
Event-Driven Ansible (EDA) [22]	Auto-remediation of misconfigured or failed infra
Prometheus / CloudWatch / Datadog	Infra monitoring and alerting for drift or violations
Compliance Scanners	Runtime compliance monitoring

E. Decommissioning Phase— Secure Resource Cleanup: Goal of this phase is to securely tear down resources and prevent data or access leakage.

a) Key Strategies:

- Automated teardown scripts: Use IaC destroy workflows to prevent orphaned resources.
- Log and access revocation: Remove credentials, secrets, and access tied to decommissioned resources.
- Retention policies and backups: Apply data retention rules securely before deletion.

b) Tools and Frameworks: "Table. V" refers to the tools and frameworks involved during IaC decommissioning. In each phase, tailored security strategies guide what needs to be protected and how.

TABLE V. TOOLS FOR DECOMMISSIONING

Tools	Purpose
Terraform Destroy / Pulumi Destroy	IaC-driven secure teardown of resources
Custom Cleanup Scripts (Ansible, Bash)	Tear down residual or orphaned cloud assets
Vault / AWS Secrets Manager	Revoke and delete secrets and credentials safely
Audit Logging Systems (CloudTrail, ELK Stack)	Verify full resource removal and access logging

V. SECURE IAC FRAMEWORK

Secure IaC Framework centered around four key pillars: Governance, Validation, Monitoring, and Resilience. These pillars work together to ensure IaC practices are secure throughout the lifecycle. "Fig. 2" provides the secure IaC framework definition.

A. Governance: This pillar focuses on establishing rules, responsibilities, and control mechanisms. Key focus areas of this pillar are

- Access Control: Enforce Role-Based Access Control (RBAC) to limit who can view or modify infrastructure code.
- Policy Enforcement: Define and manage security policies using PaC tools.
- Compliance Standards: Align templates and deployments with compliance frameworks.
- Auditing & Accountability: Maintain change logs and use signed commits for traceability.

The outcome of this pillar is to achieve a well-governed system ensuring secure design decisions and accountability.

Authorized licensed use limited to: Atlantic Technological University (ATU). Downloaded on December 14,2025 at 21:16:13 UTC from IEEE Xplore. Restrictions apply.

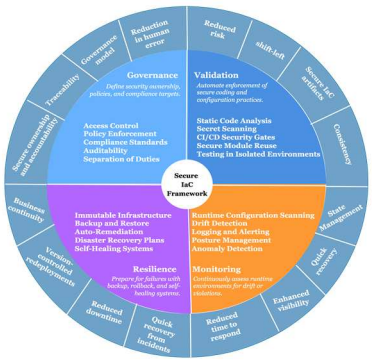

Fig. 2. Secure Iac Framework

B. Validation: This pillar focuses on validating configurations before deployment to prevent misconfigurations. Key focus areas of this pillar are

- Static Code Analysis: Use tools like Checkov or tfsec to detect issues in IaC templates.
- Secret Scanning: Ensure secrets are not exposed using tools like Gitleaks or TruffleHog.
- Test Environments: Use sandbox or ephemeral environments to validate deployments.
- CI/CD Integration: Embed security checks into pipeline.

The outcome of this pillar is to continuously validate to meet security and operational standards before it's deployed.

C. Monitoring: This pillar focuses on tracking the live environment for unauthorized changes, vulnerabilities, or policy violations. Key focus areas of this pillar are

- Drift Detection: Detect and alert when live infrastructure diverges from declared IaC (e.g., Terraform plan, AWS Config).
- Logging & Alerting: Centralize logs and set up alerts for suspicious activities using SIEM tools.
- Runtime Scanning: Use tools to assess the security posture [23] of provisioned resources in real time.
- Behavior Analysis: Monitor for unusual access patterns or activity on cloud resources.

The outcome of this pillar is ensuring the infrastructure remains compliant and secure after deployment.

D. Resilience: This pillar focuses on quick recovery, automated remediation, and minimal disruption. Key focus areas of this pillar are

- Immutable Infrastructure: Rebuild rather than patch— use versioned IaC for predictable recovery.
- Self-Healing: Trigger re-deployment or corrective actions automatically when drift or failure is detected.
- Backup & Restore: Implement secure backup strategies for critical components and state files.
- Disaster Recovery Automation: Encode recovery workflows in code for consistency and speed.

The outcome of this pillar is to make sure infrastructure is robust, recoverable, and capable of withstanding security incidents or failures. These four pillars operate in a continuous feedback loop to make sure the IaC assets are secure.

VI. Evaluation and Results

A. Evaluation Setup:

As part of this evaluation, seven cloud service teams were considered, each consisting of 30 members, that either use IaC to manage their infrastructure or IaC as their core development paradigm. Among these, about half had implemented security best practices and had automation in place—for example, running pre-commit checks or scheduling regular scans with analysis tools. A few teams had adopted a comprehensive approach, covering all recommended practices. The remaining teams, however, followed some practices manually and lacked consistency in applying the rest. While automation improved repeatability and ensured consistent practices across different environments, the focus of this analysis is on whether the security practices were followed—regardless of whether they were implemented manually or through automation. The data was gathered from each team using an Excel sheet that included a checklist of all the secure practices discussed in this paper. It also captured the impact observed both before and after these practices were implemented.

B. Results and Discussions

1) Mapping to Existing Tooling Ecosystems: Modern secure IaC frameworks leverage various solutions to detect, prevent, and remediate misconfigurations and security violations. There are existing solutions and practices like (a) PaC Tools, (b) EDA for auto-remediation, (c) IaC scanning tools, and (d) Secure State Management. The proposed framework holistically integrates core security pillars—governance, validation, resilience, and monitoring—with lifecycle goals like auto-remediation, drift detection, and secret scanning. Compared to existing tools, it acts as a conceptual blueprint that existing tools map into, ensuring secure, compliant, and automated IaC workflows.

2) Addressing IaC-Specific Attack Vectors: The proposed framework effectively addresses several known IaC-specific attack vectors through its integrated principles. Threats like Terraform state file poisoning and drift-induced vulnerabilities are mitigated through auditability, access control, and runtime drift detection. Malicious provider plugins and insecure module reuse are handled through isolated testing environments, secure CI/CD gates, and policy enforcement. Hardcoded secrets are tackled via secret scanning and separation of duties, while privilege escalation risks are reduced through static analysis and compliance-driven access controls. Overall, the framework offers a comprehensive conceptual defence that aligns closely with real-world attack surfaces, providing a strong foundation.

3) Role-Based Security Outcomes: The following "Table. VI" details the outcomes based on the individual roles involved:

TABLE VI. Role-Based Outcomes

Role	Key Metrics	Observed Improvement
Cloud Architect	% of reusable secure modules	70% increase in adoption of compliant templates
DevOps Engineer	Misconfigurations blocked predeploy	60% reduced runtime misconfigurations as the pre-approved templates are used
IaC Developer/SRE	Mean Time to Remediate (MTTR)	50% in drift-related downtime
Security Engineer	Secrets exposed in IaC repos	85% decrease in secret leaks due to the automation in place
Compliance Officer	Time to generate compliance reports	60% reduction in audit effort as the proof collection is automated and also validation is automated.

4) Holistic Security Adoption and Process Maturity: The combination of the Secure IaC Framework and lifecycle strategies created a unified security model applicable across all stages of IaC operations. Implementing this integrated approach helped with the below:

Authorized licensed use limited to: Atlantic Technological University (ATU). Downloaded on December 14,2025 at 21:16:13 UTC from IEEE Xplore. Restrictions apply.

- A 55% reduction in the effort required to maintain compliance posture.
- With tools used like tfsec, checkov, static analysis, dynamic analysis, PaC and renovate bot to raise automatic PR in git, it helped in the reduction of infrastructurerelated security incidents by 45% within 3 months.
- Measurable increase in IaC process maturity, evidenced by policy adoption and automation coverage across pipelines.

5) Toolchain Effectiveness and Automation: The integrated tool ecosystem proved essential for scaling secure IaC practices:

- Checkov, tfsec, and KICS enabled consistent IaC scanning across pipelines.
- OPA/Gatekeeper automated compliance enforcement via PaC.
- Vault and Secrets Manager ensured secret rotation and access control.
- Prisma Cloud, Driftctl, and Wiz provided runtime posture and drift visibility.

Using these tools helped teams with a 50–80% automation rate in their security tasks, greatly reducing manual reviews and remediation efforts.

6) *Secure IaC Layers and Tool Adoption Rates:* Validation tools showed the highest adoption (85%), with Checkov and tfsec integrated into 90% of CI/CD pipelines. In contrast, resilience tooling such as EDA had only 50% adoption, indicating an area for further automation maturity. "Fig. 3" provides

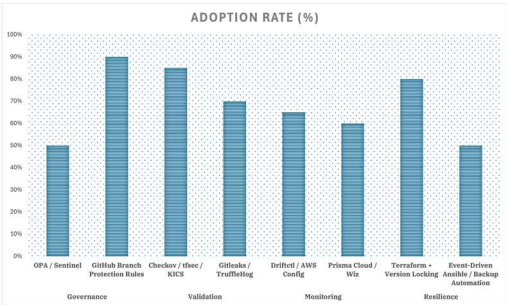

Fig. 3. Tools Adoption Rate

7) Automation impact: Around 40% of the teams studied had already implemented automation for securing their IaC assets. On average, teams reported saving approximately one person-week per month, which would otherwise have been spent performing repetitive validation, compliance checks, and remediation tasks manually.

VII. Conclusion

IaC has transformed the way organizations build and manage their infrastructure, unlocking speed and scalability like never before. Yet with this evolution comes a new wave of security risks—ones that can no longer be addressed through traditional means. This paper explored a comprehensive set of strategies that empower organizations to secure their digital assets by design, not by afterthought. The proposed framework helps teams to integrate security early in the lifecycle and assign the responsibility to the right team members to achieve the security of digital assets. This research measured the effectiveness of the proposed framework in terms of maturity and the reduction in effort. This research also concludes that consistent tool adoption helped the teams to automate security scans. As the digital landscape grows more dynamic and complex, the models and frameworks presented in this research offer a practical roadmap for organizations to build secure, resilient, and compliant infrastructure—not just for today, but for whatever comes next.

References

[1] Akond Rahman, Rezvan Mahdavi-hezaveh and Laurie Williams, "A Systematic Mapping Study of Infrastructure as Code Research" Information and Software Technology Volume 108,pp.65-77, April 2019.

[2] A. Rahman, F. L. Barsha and P. Morrison, "Shhh!: 12 Practices for Secret Management in Infrastructure as Code," 2021 IEEE Secure Development Conference (SecDev), Atlanta, GA, USA, pp. 56-62, 2021.

[3] J. Alonso, R. Piliszek and M. Cankar, "Embracing IaC Through the DevSecOps Philosophy: Concepts, Challenges, and a Reference Framework," in IEEE Software, vol. 40, no. 1, pp. 56-62, Jan.-Feb. 2023.

[4] Akshay Nagpal, et al., "Framework for automating compliance verification in CI/CD pipelines,", International Journal of Computer Science and Information Technology Research 2024, Vol. 5, No. 4, OctoberDecember, pp. 17-27, 2024.

[5] Kyadasu, R., Dave, A., Arulkumaran, R., Goel, O., Kumar, D. L., Jain, P. A. (2024). "Exploring Infrastructure as Code Using Terraform in Multi-Cloud Deployments." Journal of Quantum Science and Technology (JQST), 1(4), Nov 2024.

[6] A. Zeini, R. G. Lennon and P. Lennon, "Securing Infrastructure as Code (IaC) through DevSecOps:A Comprehensive Risk Management Framework," 2023 Cyber Research Conference - Ireland (Cyber-RCI), Letterkenny, Ireland, pp. 1-11, 2023.

[7] Ramreddy Gouni, "Automating Compliance In Devops Pipelines", International Journal of Computational and Experimental Science and Engineering 11(2), April 2025.

[8] MIchele Guerriero, Martin Garriga and Damian A. Tamburri, "Adoption, Support, and Challenges of Infrastructure-as-Code: Insights from Industry", 2019 IEEE International Conference on Software Maintenance and Evolution (ICSME), 2019.

[9] A. Rahman and L. Williams, "Different Kind of Smells: Security Smells in Infrastructure as Code Scripts," in IEEE Security & Privacy, vol. 19, no. 3, pp. 33-41, May-June 2021.

[10] Akond Rahman, Effat Farhana, Laurie Williams "The 'as code' activities: development anti-patterns for infrastructure as code" Empirical Software Engineering, Volume 25, pages 3430–3467, (2020).

[11] Michele Chiari, Michele De Pascalis, and Matteo Pradella "Static Analysis of Infrastructure as Code: a Survey" 2022 IEEE 19th International Conference on Software Architecture Companion (ICSA-C), 2022.

[12] Akond Rahman, Chris Parnin, Laurie Williams "The Seven Sins: Security Smells in Infrastructure as Code Scripts", 2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE), 2019.

[13] Aicha War, Andrew Habib, Alioune Diallo and Jacques Klein "Security Vulnerabilities in Infrastructure as Code: What, How Many, and Who?", Empirical Software Engineering, Volume 30, article number 120, 05 June 2025

[14] Sadiq Almuairfi, Mamdouh Alenezi, "Security controls in infrastructure as code," Computer Fraud & Security, Volume 2020, Issue 10, Pages 13-19, 2020.

[15] J. Alonso, R. Piliszek and M. Cankar, "Embracing IaC Through the DevSecOps Philosophy: Concepts, Challenges, and a Reference Framework," in IEEE Software, vol. 40, no. 1, pp. 56-62, Jan.-Feb. 2023.

[16] Plachkinova, M., & Knapp, K., Least Privilege across People, Process, and Technology: Endpoint Security Framework. Journal of Computer Information Systems, 63(5), 1153–1165, 2022.

[17] Sarathe Krisshnan Jutoo Vijayaraghavan, "Security as code: Transforming DevSecOps through CI/CD Integration", World Journal of Advanced Engineering Technology and Sciences, 15(01), 2219-2225, 2025.

[18] S. K. Basak, J. Cox, B. Reaves and L. Williams, "A Comparative Study of Software Secrets Reporting by Secret Detection Tools," 2023 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM), New Orleans, LA, USA, pp. 1-12, 2023.

[19] R. Singh, A. Yeboah-Ofori, S. Kumar and A. Ganiyu, "Fortifying Cloud DevSecOps Security Using Terraform Infrastructure as Code Analysis Tools," 2024 International Conference on Electrical and Computer Engineering Researches (ICECER), Gaborone, Botswana,pp. 1-6, 2024.

[20] J. Liang et al., "Code as Policies: Language Model Programs for Embodied Control," 2023 IEEE International Conference on Robotics and Automation (ICRA), London, United Kingdom,pp. 9493-9500, 2023.

[21] Gogulakrishnan Thiyagarajan, Vinay Bist, Prabhudarshi Nayak, "AIDriven Configuration Drift Detection in Cloud Environments",

Authorized licensed use limited to: Atlantic Technological University (ATU). Downloaded on December 14,2025 at 21:16:13 UTC from IEEE Xplore. Restrictions apply.

International Journal of Communication Networks and Information Security 16(05):721-743, January 2024.
[22] Brendan Choi, Erwin Medina, ""Is Ansible Good for Network Automation?", Introduction to Ansible Network Automation, October 2023.
[23] Huang, K., Li, Y., Thaine, P. "Use GenAI Tools to Boost Your Security Posture", Generative AI Security (pp.305-338), April 2024.

Authorized licensed use limited to: Atlantic Technological University (ATU). Downloaded on December 14,2025 at 21:16:13 UTC from IEEE Xplore. Restrictions apply.